

First-Order Entropy Balancing via Dual Gradient Descent^{*}

Gaurav Sood[†]

January 2, 2026

Abstract

Entropy balancing constructs calibration weights by minimizing Kullback–Leibler divergence from base weights subject to linear moment constraints. Because the primal problem admits a smooth convex dual, common software solves the dual using Newton or Levenberg–Marquardt type methods, or quasi-Newton approximations, which can be effective when the number of constraints is moderate but can become costly as the number of constraints grows. We study plain gradient descent on the standard entropy balancing dual and show that, through the exponential-tilting map, each dual step induces a multiplicative (exponentiated) adjustment of the implied primal weights. This correspondence is standard in entropic mirror descent, but making it explicit in the entropy balancing setting yields a transparent solver with cheap iterations, simple warm starts when targets are revised, and diagnostics directly tied to moment error. Under bounded features and standard smooth convex conditions, the dual objective gap decreases at rate $O(1/T)$, while a worst-case stationarity guarantee yields moment error $O(1/\sqrt{T})$ for the best iterate, with faster rates possible under additional regularity. We compare accuracy and runtime against quasi-Newton and Levenberg–Marquardt baselines in synthetic experiments, and we illustrate why naive mini-batch approximations are generally biased for this dual objective due to its global log-sum-exp normalization.

1 Introduction

Consider a sample of n observations with feature vectors $x_1, \dots, x_n \in \mathbb{R}^d$ and population targets $\bar{x}_{\text{pop}} \in \mathbb{R}^d$. The calibration problem is to find weights $w_1, \dots, w_n$ that sum to one

^{*}Replicated materials are posted at <https://github.com/finite-sample/adaptive-eb>

[†]Gaurav can be reached at contact@gsood.com

and satisfy

$$\sum_{i=1}^n w_i x_i = \bar{x}_{\text{pop}}.$$

The features may include raw covariates as well as indicator expansions and interactions chosen to capture important differences between the sample and the population. Calibration weighting is widely used in survey research and causal inference, with close ties to generalized calibration and raking (Deville and Särndal, 1992; Deville et al., 1993), iterative proportional fitting (Deming and Stephan, 1940; Ireland and Kullback, 1968), and covariate balancing propensity score methods (Imai and Ratkovic, 2014).

Entropy balancing provides a principled formulation of calibration (Hainmueller, 2012). Given base weights u (often uniform weights or survey design weights), entropy balancing chooses weights that minimize Kullback–Leibler divergence from u subject to the moment constraints. The resulting weights admit an exponential-tilting representation and can be computed by solving an unconstrained dual problem.

Computationally, many entropy balancing implementations solve the dual using curvature information via Newton or Levenberg–Marquardt type steps, or quasi-Newton approximations (Hainmueller and Xu, 2013). These methods can converge in relatively few iterations when d is moderate, but their per-iteration work involves computations in $\mathbb{R}^{d \times d}$: in dense settings, forming weighted covariance information can cost $O(nd^2)$ and solving a linear system can cost $O(d^3)$, while quasi-Newton methods avoid explicit Hessians but typically incur $O(d^2)$ state and line-search evaluations. These considerations matter when calibration uses many categories and interactions, or when recalibration must be repeated as targets are revised.

This paper studies a basic first-order method for the entropy balancing dual. We show that gradient descent on the dual induces multiplicative, exponentiated updates of the implied primal weights. This is a standard phenomenon in mirror descent with a negative-entropy geometry, and it provides a simple algorithmic lens for entropy balancing that makes warm starts and diagnostics particularly transparent.

We make four contributions. First, we present a self-contained derivation of a dual first-order method for entropy balancing and the induced multiplicative update for primal weights (Section 4). Second, we provide convergence guarantees that distinguish objective progress from moment error (Section 4.4). Third, we discuss implementation choices that affect stability and speed, including feature preprocessing, step sizes, numerical stability, and warm starts (Section 4.5). Fourth, we compare accuracy and runtime against common second-order baselines in experiments (Section 5).

Throughout, the sample is fixed. Iterations refer to optimization steps, not to

sequential arrival of observations.

2 Related Work

Entropy balancing is a member of the broader class of calibration methods that construct weights by minimizing a convex discrepancy from base (e.g., design) weights subject to linear moment constraints (Dewille and Särndal, 1992; Dewille et al., 1993). In the special case of contingency-table margin constraints (i.e., indicator expansions of categorical variables), iterative proportional fitting (also known as raking) computes the Kullback–Leibler projection through successive multiplicative adjustments (Deming and Stephan, 1940; Ireland and Kullback, 1968). More generally, Csiszár (1975) studies I -divergence (KL) projections and iterative projection algorithms, and generalized iterative scaling provides a closely related multiplicative procedure for maximum-entropy/log-linear models (Darroch and Ratcliff, 1972). Entropy balancing, introduced by Hainmueller (2012), fits squarely in this maximum-entropy calibration tradition and is closely related to exponential tilting representations of calibration weights.

In causal inference, entropy balancing is part of a broader shift toward methods that enforce covariate balance through explicit moment conditions. Subsequent theoretical work establishes properties such as double robustness under standard conditions (Zhao and Percival, 2017). Related approaches impose balance via propensity-score estimation with balancing conditions (Imai and Ratkovic, 2014) or construct balancing weights through alternative convex criteria designed to control weight variability (Zubizarreta, 2015).

On computation, many widely used entropy balancing implementations exploit the smooth, unconstrained dual formulation and apply Newton-type updates (including Levenberg–Marquardt variants) that use second-order information for rapid convergence when the number of constraints is moderate (Hainmueller and Xu, 2013). The cost of these methods can rise with the number of balance constraints because each iteration involves curvature computations and linear-system solves in dimension d , which can become burdensome in high-dimensional calibrations or in workflows requiring repeated recalibration. Motivated by such settings, recent work and software have explored alternative solvers and scalable implementations, including dual-based calibration libraries with support for weight bounds and inexact calibration (Wang et al., 2019) and distributed gradient-based solvers for entropy balancing at large scale (Lin et al., 2023).

The algorithmic perspective in this paper builds on first-order methods for en-

tropic objectives, where negative-entropy geometry yields multiplicative/exponentiated updates. This includes exponentiated-gradient algorithms (Kivinen and Warmuth, 1997), multiplicative-weights frameworks (Arora et al., 2012), and mirror descent with entropic regularization (Beck and Teboulle, 2003; Bubeck, 2015). Our contribution is to make the induced multiplicative structure explicit in the entropy balancing dual, and to provide convergence guarantees and implementation guidance tailored to calibration applications.

3 Background

This section reviews entropy balancing (EB) as a convex calibration problem and introduces the dual formulation that underlies essentially all practical solvers.

3.1 Entropy Balancing

Calibration weighting starts from the viewpoint that a set of weights defines a *distribution over the sample units*. Let $x_i \in \mathbb{R}^d$ denote a vector of features for observation i (raw covariates, indicator expansions, interactions, or other balance functions), and let $\bar{x}_{\text{pop}} \in \mathbb{R}^d$ denote externally supplied population targets (e.g., population means or margins). We also assume a set of *base weights* u that encode the weighting scheme in the absence of calibration—for example, uniform weights, sampling/design weights, or previously computed weights.

Entropy balancing chooses new weights w that (i) exactly reproduce the target moments and (ii) deviate from the base weights as little as possible, where deviation is measured by Kullback–Leibler (KL) divergence. Formally, with $\Delta^{n-1} = \{w \in \mathbb{R}_+^n : \sum_i w_i = 1\}$, EB solves

$$\min_{w \in \Delta^{n-1}} \text{KL}(w \| u) \quad \text{subject to} \quad \sum_{i=1}^n w_i x_i = \bar{x}_{\text{pop}}, \quad (\text{EB})$$

where $\text{KL}(w \| u) = \sum_i w_i \log(w_i / u_i)$ and $u_i > 0$ for all i . This objective has a clear interpretation: among all weighting schemes that satisfy the balance constraints, EB selects the one that is closest to u in relative-entropy terms. When u is uniform, EB is equivalent to choosing the *maximum-entropy* distribution over units subject to the moment constraints; when u reflects a prior weighting scheme (e.g., survey design weights), EB produces the smallest KL adjustment that enforces calibration (Deville and Särndal, 1992; Csiszár, 1975; Hainmueller, 2012).

Two basic structural facts are worth keeping in mind. First, the feasible set is defined by linear equalities intersected with the simplex, so feasibility requires that $\bar{x}_{\text{pop}}$ lie in the convex hull of $\{x_1, \dots, x_n\}$ (under nonnegative weights summing to one). Second, whenever the constraints are feasible, the KL objective is strictly convex in w on the simplex, so the entropy balancing solution is unique (Csiszár, 1975; Hainmueller, 2012). These properties make EB a natural default when one wants *exact balance* without introducing unnecessary weight volatility beyond what the constraints force.

3.2 Dual Formulation

A central reason EB is computationally attractive is that it admits a smooth, unconstrained dual problem. The standard derivation proceeds by forming the Lagrangian with multipliers $\lambda \in \mathbb{R}^d$ for the moment constraints (and an additional multiplier for the simplex normalization). Solving the first-order optimality conditions yields an *exponential tilting* (or Gibbs) form for the primal weights (Hainmueller, 2012):

$$w_i(\lambda) = \frac{u_i \exp(-\lambda^\top x_i)}{Z(\lambda)}, \quad Z(\lambda) = \sum_{j=1}^n u_j \exp(-\lambda^\top x_j). \quad (1)$$

The normalizing constant $Z(\lambda)$ ensures the weights sum to one. This representation makes the economic interpretation of λ transparent: each component of λ acts like a “shadow price” that penalizes or rewards units based on their feature values so that the weighted feature averages match the targets.

Substituting (1) into the Lagrangian yields the convex dual objective:

$$D(\lambda) = \log Z(\lambda) + \lambda^\top \bar{x}_{\text{pop}}, \quad (2)$$

and the dual problem is to minimize $D(\lambda)$ over $\lambda \in \mathbb{R}^d$. The function $\log Z(\lambda)$ is the log-partition (log-sum-exp) term associated with the tilted weights, which is smooth and convex; the linear term $\lambda^\top \bar{x}_{\text{pop}}$ encodes the calibration targets. The gradient takes an especially intuitive form:

$$\nabla D(\lambda) = \bar{x}_{\text{pop}} - \sum_{i=1}^n w_i(\lambda) x_i. \quad (3)$$

Thus, $\nabla D(\lambda)$ is exactly the discrepancy between the target moments and the moments implied by the current tilted weights. Minimizing $D(\lambda)$ (equivalently, finding a zero of its gradient) is therefore the same as finding λ such that the implied weights satisfy the calibration constraints.

For later use, define the moment error

$$g(\lambda) := \sum_{i=1}^n w_i(\lambda) x_i - \bar{x}_{\text{pop}},$$

so that $\nabla D(\lambda) = -g(\lambda)$.

Finally, the curvature of the dual is also interpretable: the Hessian is the covariance matrix of the features under the current weights,

$$\nabla^2 D(\lambda) = \text{Cov}_{w(\lambda)}[x].$$

This identity makes clear why second-order methods can be effective (they exploit feature covariance under the evolving weights), and also why curvature computations can become expensive as d grows. It also provides useful intuition for first-order methods: directions with high weighted variance correspond to directions of higher curvature in the dual objective, and hence tend to dictate conservative step sizes for stable descent.

Feasibility and boundary behavior. Feasibility requires $\bar{x}_{\text{pop}}$ to lie in the convex hull of $\{x_1, \dots, x_n\}$ under simplex weights. Even when feasible, targets near the boundary can induce highly concentrated optimal weights. In such cases, primal solutions may exist while the dual infimum may fail to be attained at a finite λ ; optimization can still drive the residual toward zero while $\|\lambda\|$ grows. We return to diagnostics and practical responses to infeasibility and boundary behavior in [Section 4.5](#).

4 First-Order Entropy Balancing

This section describes the optimization procedure studied in the paper. The dual objective $D(\lambda)$ in (2) is smooth and convex, and its gradient (3) is exactly the gap between the target moments and the moments implied by the current weights. As a result, minimizing $D(\lambda)$ is equivalent to driving the calibration error to zero, and a natural approach is to apply first-order methods that require only repeated evaluations of weighted moments.

4.1 Dual gradient descent

We apply gradient descent to the unconstrained dual problem:

$$\lambda^{(t+1)} = \lambda^{(t)} - \eta \nabla D(\lambda^{(t)}), \tag{4}$$

with step size $\eta > 0$. Using $\nabla D(\lambda) = \bar{x}_{\text{pop}} - \sum_i w_i(\lambda) x_i$ from (3), it is convenient to work with the moment error

$$g(\lambda) := \sum_{i=1}^n w_i(\lambda) x_i - \bar{x}_{\text{pop}},$$

so that $\nabla D(\lambda) = -g(\lambda)$ and

$$\lambda^{(t+1)} = \lambda^{(t)} + \eta g^{(t)}, \quad g^{(t)} := g(\lambda^{(t)}). \quad (5)$$

This form makes the direction of the update transparent: if a feature is currently overrepresented under $w(\lambda^{(t)})$ relative to $\bar{x}_{\text{pop}}$, the corresponding component of $g^{(t)}$ is positive and the next step increases the associated dual coordinate, which (through exponential tilting) downweights units with large values of that feature.

4.2 Induced multiplicative reweighting

Although (5) is written in the dual variables, it implies a simple multiplicative update on the primal weights. Let $w^{(t)} := w(\lambda^{(t)})$ denote the weights induced by (1).

Lemma 1 (Induced multiplicative update). *For any iterate $\lambda^{(t)}$ and step size $\eta > 0$, the dual update $\lambda^{(t+1)} = \lambda^{(t)} + \eta g^{(t)}$ implies the primal update*

$$w_i^{(t+1)} \propto w_i^{(t)} \exp(-\eta x_i^\top g^{(t)}), \quad (6)$$

where the proportionality constant is the normalization that enforces $\sum_i w_i^{(t+1)} = 1$.

Proof. By (1),

$$w_i^{(t+1)} \propto u_i \exp(-\lambda^{(t+1)\top} x_i) = u_i \exp(-\lambda^{(t)\top} x_i) \exp(-\eta x_i^\top g^{(t)}) \propto w_i^{(t)} \exp(-\eta x_i^\top g^{(t)}).$$

□

Equation (6) provides an intuitive interpretation of each iteration. The vector $g^{(t)}$ summarizes which moments are too large or too small under the current weights. Each unit i is then adjusted according to the alignment between its feature vector x_i and the imbalance direction $g^{(t)}$. Units that contribute heavily to overrepresented features are downweighted, and units that contribute to underrepresented features are upweighted. The exponential form ensures weights remain positive and naturally preserves the simplex constraint after renormalization.

4.3 Stable evaluation, stopping, and cost

In implementation it is numerically safer to evaluate weights via stabilized log-weights rather than by repeatedly multiplying (6) across iterations. Given λ , define the unnormalized log-weights

$$\ell = \log u - X\lambda \in \mathbb{R}^n,$$

and apply the standard log-sum-exp stabilization $\ell \leftarrow \ell - \max_i \ell_i$ before exponentiating and normalizing. This avoids overflow/underflow when $\|\lambda\|$ grows.

A natural stopping rule is based on the calibration residual. Since $g(\lambda) = X^\top w(\lambda) - \bar{x}_{\text{pop}}$, we terminate when $\|g(\lambda)\|_\infty$ (or another norm) falls below a user-specified tolerance.

Each iteration requires (i) computing scores $X\lambda$ to form the weights and (ii) computing the weighted moments $X^\top w$. With sparse X these are $O(\text{nnz}(X))$ operations per iteration; with dense X they are $O(nd)$. Beyond storing X , the algorithm stores $w \in \mathbb{R}^n$, $\lambda \in \mathbb{R}^d$, and a small number of scratch vectors. Warm starts are immediate: one can initialize $\lambda^{(0)}$ from a previous run (e.g., after revising targets) and continue the same iterations.

Input: $X \in \mathbb{R}^{n \times d}$ (row i is $x_i^\top$), targets $\bar{x}_{\text{pop}} \in \mathbb{R}^d$, base weights $u \in \Delta^{n-1}$ with

$u_i > 0$, step size $\eta > 0$, tolerance $\tau > 0$, max iterations T_{max}

Output: Calibration weights $w \in \Delta^{n-1}$

Initialize $\lambda \leftarrow 0$;

for $t = 0, 1, \dots, T_{\text{max}} - 1$ **do**

$\ell \leftarrow \log u - X\lambda$;

$\ell \leftarrow \ell - \max_i \ell_i$; // stabilize

$w \leftarrow \exp(\ell)$; $w \leftarrow w / \sum_{i=1}^n w_i$;

$g \leftarrow X^\top w - \bar{x}_{\text{pop}}$;

if $\|g\|_\infty \leq \tau$ **then return** w ;

$\lambda \leftarrow \lambda + \eta g$;

end

return w ;

Algorithm 1: First-order entropy balancing via dual gradient descent

The next section specializes standard smooth convex optimization results to $D(\lambda)$ to obtain rates for objective decrease and moment error as a function of the iteration budget.

4.4 Convergence guarantees

The dual objective $D(\lambda)$ in (2) is smooth and convex, so gradient descent admits standard objective and stationarity guarantees (Nesterov, 2018). In calibration, however, it is useful to distinguish two notions of progress: (i) decrease in the dual objective value, and (ii) satisfaction of the moment constraints. These are related but not identical. The key observation is that the dual gradient is (the negative of) the moment error:

$$\nabla D(\lambda) = \bar{x}_{\text{pop}} - X^\top w(\lambda) = -g(\lambda),$$

so controlling $\|\nabla D(\lambda)\|$ is exactly controlling imbalance.

Proposition 1 (Objective gap and moment error for dual gradient descent). *Assume $\|x_i\|_2 \leq B$ for all i , and assume the dual problem attains its minimum at some $\lambda^* \in \mathbb{R}^d$.¹ Then D has an L -Lipschitz gradient (Euclidean norm) with*

$$L = \sup_{\lambda} \|\nabla^2 D(\lambda)\|_{\text{op}} \leq B^2.$$

Run gradient descent with constant step size $\eta \leq 1/L$: $\lambda^{(t+1)} = \lambda^{(t)} - \eta \nabla D(\lambda^{(t)})$. Then for all $T \geq 1$,

$$D(\lambda^{(T)}) - D(\lambda^*) \leq \frac{\|\lambda^{(0)} - \lambda^*\|_2^2}{2\eta T}, \quad (7)$$

$$\min_{0 \leq t \leq T-1} \|g(\lambda^{(t)})\|_2 = \min_{0 \leq t \leq T-1} \|\nabla D(\lambda^{(t)})\|_2 \leq \sqrt{\frac{2(D(\lambda^{(0)}) - D(\lambda^*))}{\eta T}}. \quad (8)$$

Proof. See Appendix A.

Equation (7) is the standard $O(1/T)$ objective-gap guarantee for smooth convex optimization. Equation (8) is a worst-case stationarity bound, stated for the best iterate along the path; it directly controls moment error because the dual gradient is exactly the imbalance vector. If a stopping rule uses an ℓ_∞ tolerance (common in calibration), note that $\|g\|_\infty \leq \|g\|_2$, so the ℓ_2 bound implies an ℓ_∞ guarantee as well.

Faster (geometric) convergence is possible under additional regularity. In particular, if the dual is strongly convex on a set containing the iterates—i.e., if there exists $\mu > 0$ such that $\nabla^2 D(\lambda) \succeq \mu I$ throughout that set—then gradient descent with a suitable step size converges linearly in objective gap and gradient norm. For entropy balancing this requires the weighted feature covariance $\text{COV}_{w(\lambda)}[x]$ to be uniformly well-conditioned

¹If targets lie on the boundary of the feasible moment set, the dual infimum may fail to be attained at a finite λ^* even when primal solutions exist; see the discussion in Section 3 and Section 4.5.

along the optimization path. The condition may fail when features are (nearly) collinear or when the iterates induce highly concentrated weights, so it is best treated as an additional assumption rather than a baseline guarantee.

Remark 1 (Relation to multiplicative-weights regret bounds). *Multiplicative-weights analyses in online learning typically prove $O(1/\sqrt{T})$ regret bounds against adversarial sequences (Arora et al., 2012). Those results control cumulative performance relative to a comparator and address a different problem class. Here we solve a fixed smooth convex minimization problem, and Proposition 1 provides the relevant objective and moment-error guarantees for that setting.*

4.5 Implementation considerations

This subsection collects choices that matter for numerical stability, iteration count, and interpretability. The algorithmic update is fully specified in Section 4 (Algorithm 1); the points below address how to make the method reliable in common calibration regimes.

4.5.1 Feature construction, identifiability, and conditioning

The balance functions x_i determine both the estimand being calibrated to and the geometry of the dual problem. Richer features (e.g., indicator expansions, interactions, nonlinear transforms) can reduce observed imbalance but may also induce more variable weights and can worsen conditioning.

Two practical issues show up repeatedly:

Redundancy and rank deficiencies. If the columns of X are linearly dependent (e.g., a full set of category indicators together with an intercept implied by $\sum_i w_i = 1$), the dual may be non-identifiable or poorly conditioned. In practice this manifests as slow progress and large $\|\lambda\|$ with only small changes in weights. Removing redundant columns (e.g., dropping one indicator per categorical variable, or explicitly checking for near-collinearity) improves both stability and interpretability.

Centering and scaling. Centering and scaling help align numerical behavior with theoretical step-size bounds. A useful transformation is to replace x_i by $x_i - \bar{x}_{\text{pop}}$, which turns the target into zero. This is equivalent to the original constraints because weights are always normalized ($\sum_i w_i = 1$). Scaling continuous features to comparable magnitudes

reduces sensitivity to units and makes a single global step size behave more uniformly across coordinates.

4.5.2 Step size selection and monotonicity

The convergence guarantees require $\eta \leq 1/L$, where L is any valid Lipschitz constant for ∇D . Under the bounded-feature assumption in Proposition 1, $L \leq \max_i \|x_i\|_2^2$ is a conservative sufficient bound, so

$$\eta = \frac{1}{\max_i \|x_i\|_2^2}$$

is a safe default when features are not standardized.

In well-scaled problems, larger fixed step sizes can work, but overly aggressive η may produce oscillations in moment error or increases in $D(\lambda)$. A simple and robust safeguard is to monitor $D(\lambda^{(t)})$: for $\eta \leq 1/L$, smoothness implies a descent property, so persistent increases suggest that the step size is too large or that numerical error is dominating.

Coordinate-adaptive schedules (e.g., AdaGrad (Duchi et al., 2011)) can be useful when feature magnitudes differ substantially across coordinates, but they change the update rule and should be viewed as a robustness heuristic rather than the specific method analyzed in Proposition 1.

4.5.3 Stopping criteria and diagnostics

A natural stopping criterion is based on the moment residual $g(\lambda) = X^\top w(\lambda) - \bar{x}_{\text{pop}}$. The ℓ_∞ criterion

$$\|g(\lambda^{(t)})\|_\infty \leq \tau$$

is often the most interpretable in calibration work because it controls the maximum absolute constraint violation. Since $\|g\|_\infty \leq \|g\|_2$, it is also compatible with the ℓ_2 -based theory.

In addition to the residual, two diagnostics are routinely informative:

Dual objective tracking. When η is suitably chosen and computations are stable, $D(\lambda^{(t)})$ should typically decrease. This is a sensitive indicator of step-size issues.

Weight concentration. Calibration can succeed numerically while producing highly concentrated weights. Reporting the effective sample size $\text{ESS} = 1 / \sum_i w_i^2$, the maximum weight, and weight quantiles helps separate “balance achieved” from “balance achieved with acceptable variance inflation.”

4.5.4 Numerical stability and feasibility signals

Weights should be computed from stabilized log-weights via the log-sum-exp trick, as in Algorithm 1. This avoids overflow/underflow when $\lambda^\top x_i$ varies widely.

If the specified targets are infeasible (e.g., outside the convex hull of $\{x_i\}$ under simplex weights), exact calibration is impossible. In such cases, first-order iterations often drive weights toward extreme concentration and $\|\lambda\|$ may grow without achieving small residuals. When this pattern appears, it is typically preferable to modify the problem (e.g., relax constraints, introduce penalties, or impose weight bounds) rather than to force the exact constraints through numerical tolerances.

4.5.5 Weight control and relaxed calibration

Some applications require explicit control of weight variability (for variance or robustness reasons). This can be achieved by modifying the optimization problem—for example, by adding regularization that penalizes dispersion or by allowing approximate moment matching. Such variants are studied in the balancing-weights literature (Zubizarreta, 2015) and in calibration frameworks that permit inexact balance or additional constraints (Wang et al., 2019). These modifications change the dual objective and therefore fall outside the exact guarantees stated in Proposition 1, but they provide principled alternatives when exact EB yields undesirable weight concentration.

4.5.6 Warm starts and revised targets

Warm starts are immediate in the dual. When targets are revised (e.g., benchmarking updates or sensitivity analysis), initializing $\lambda^{(0)}$ at the previous solution and continuing the same iterations typically reduces the number of steps needed to reach the new tolerance, especially when the new targets are close to the old ones.

4.5.7 Large-scale computation

For large n , the dominant costs are repeated multiplications by X and $X^\top$ to compute scores and weighted moments. Sparse representations and optimized linear algebra

usually deliver the largest gains. Distributed systems often retain the same dual formulation but differ in how they parallelize these passes over the data and how they schedule first-order steps (Lin et al., 2023). Approximate moment computations are possible, but because $g(\lambda)$ is a weighted expectation under the *current* $w(\lambda)$, approximation schemes should be designed to control bias/variance and to preserve correct normalization.

5 Experiments

We evaluate three questions. First, can full-gradient dual gradient descent (dual GD) recover the same entropy balancing solution as a standard quasi-Newton solver on feasible problems? Second, how does runtime change with dimensionality and with problem difficulty (as measured by the concentration of the implied weights)? Third, do two practical variants matter in the intended use cases: warm starts under revised targets and approximate mini-batch updates?

To evaluate these, our simulation setup is as follows: each replicate draws bounded features and constructs targets that are feasible by design. We generate $X \in \mathbb{R}^{n \times d}$ with entries $X_{ij} = \tanh(Z_{ij})$ where $Z_{ij} \sim \mathcal{N}(0, 1)$, so $X_{ij} \in (-1, 1)$. Base weights are uniform, $u_i = 1/n$.

To simulate easy targets, we draw simplex weights $a \sim \text{Dirichlet}(\alpha \mathbf{1}_n)$ over all n points and set $\bar{x}_{\text{pop}} = X^\top a$. This produces feasible targets that typically do not require large departures from uniform weights.

To induce more concentrated solutions, we first draw a direction $v \in \mathbb{R}^d$ and select the k observations with largest projection Xv . We then draw weights a supported only on those k points via $a_{\text{sub}} \sim \text{Dirichlet}(\alpha \mathbf{1}_k)$, embed into $\mathbb{R}^n$ (zeros elsewhere), and set $\bar{x}_{\text{pop}} = X^\top a$. This remains feasible by construction but pushes the target toward the boundary of the convex hull, typically lowering the effective sample size (ESS).

Across these simulated scenarios, we compare:

1. **Dual BFGS (baseline).** We minimize the dual objective with `scipy.optimize.minimize` using BFGS and analytic gradients.
2. **Dual GD (batch).** We run full gradient descent on the dual and recover weights via exponential tilting. Unless otherwise noted, we use the conservative step size $\eta = 1/\max_i \|x_i\|_2^2$ after scaling features.
3. **Levenberg–Marquardt (LM) / nonlinear least squares baseline.** We solve the moment equations by minimizing $\frac{1}{2}\|g(\lambda)\|_2^2$ using a Levenberg–Marquardt style method

(or a standard nonlinear least-squares routine) with analytic Jacobian $J(\lambda) = \nabla g(\lambda) = -\nabla^2 D(\lambda) = -\text{COV}_{w(\lambda)}[x]$ where feasible. This baseline is closer to common entropy balancing implementations (Hainmueller and Xu, 2013).

4. **Dual GD (mini-batch, approximate).** We implement a naive approximation that self-normalizes weights within each mini-batch and updates using the resulting approximate moment error.
5. **Warm start under revised targets.** We revise the target via a convex combination of two feasible targets and compare continuing from the prior dual solution to restarting from $\lambda = 0$.

We report $\|g\|_2$ where $g = X^\top w - \bar{x}_{\text{pop}}$, runtime in seconds, ESS defined as $1/\sum_i w_i^2$, and optimization iterations. Full-gradient methods stop when $\|g\|_\infty < \tau$ or a maximum iteration cap is reached.

Because BFGS iterations are not directly comparable to gradient steps, we also report BFGS evaluation counts (objective evaluations nfev and gradient evaluations njev) to contextualize runtime.

5.1 Results

In the standard and high-dimensional easy settings, dual GD and BFGS reach moment error at the stated tolerances and produce ESS values close to n (see Table 1). This indicates that the implied weights remain close to uniform in these regimes (see Figure 1). In these runs, dual GD is faster in wall-clock time despite using comparable or slightly more iterations, consistent with its low per-iteration overhead.

The other thing to note is that the mini-batch variant stabilizes far above the stringent tolerances achieved by full-gradient methods. This behavior reflects the systematic bias inherent in the naive mini-batch approximation. Unlike standard SGD, where subsampled gradients are unbiased estimators of the true gradient, the dual gradient in entropy balancing depends on a global normalization factor $Z(\lambda) = \sum \exp(-\lambda^\top x_i)$. Computing the normalization locally on a mini-batch results in a biased gradient estimator; the algorithm effectively optimizes a different objective—the sum of mini-batch duals—rather than the true global dual, leading to a noise floor that does not vanish with additional epochs.

In the hard-target regime, ESS drops sharply (see Figure 1), indicating a substantially more concentrated solution. Both methods still reach the target tolerance, but the number of iterations and runtime increase materially. In this regime, the relative

Table 1. Performance comparison (mean $\pm$ SE over seeds).

Method	$\ g\ _2$	Time (s)	ESS	Iterations
<i>Standard (easy targets): $n = 3000, d = 10$</i>				
BFGS	$(1.06 \pm 0.00) \times 10^{-8}$	1.063 ± 0.000	2992.6 ± 0.0	8.0 ± 0.0
Dual GD (Batch)	$(9.23 \pm 0.00) \times 10^{-9}$	0.898 ± 0.000	2992.6 ± 0.0	9.0 ± 0.0
Dual GD (Mini-batch)	0.002 ± 0.000	0.311 ± 0.000	2992.6 ± 0.0	600.0 ± 0.0
<i>High-dimensional (easy targets): $n = 3000, d = 100$</i>				
BFGS	$(2.10 \pm 0.00) \times 10^{-6}$	3.691 ± 0.000	2894.2 ± 0.0	11.0 ± 0.0
Dual GD (Batch)	$(1.74 \pm 0.00) \times 10^{-6}$	1.000 ± 0.000	2894.2 ± 0.0	14.0 ± 0.0
<i>Standard (hard targets): $n = 1000, d = 10$</i>				
BFGS	$(1.16 \pm 0.00) \times 10^{-6}$	3.400 ± 0.000	17.0 ± 0.0	31.0 ± 0.0
Dual GD (Batch)	$(9.61 \pm 0.00) \times 10^{-7}$	5.901 ± 0.000	17.0 ± 0.0	119.0 ± 0.0

Table 2. BFGS evaluation counts (mean $\pm$ SE over seeds).

Scenario	Iterations	Obj evals (nfev)	Grad evals (njev)
Standard (easy targets)	8.0 ± 0.0	9.0 ± 0.0	9.0 ± 0.0
High-dimensional (easy targets)	11.0 ± 0.0	12.0 ± 0.0	12.0 ± 0.0
Standard (hard targets)	31.0 ± 0.0	32.0 ± 0.0	32.0 ± 0.0

speed advantage of dual GD is reduced and can reverse, reflecting the larger number of first-order steps needed when the effective conditioning along the optimization path is less favorable.

Moving to comparison between warm and cold starts, after modest target revisions, warm starts typically reduce the number of additional iterations needed to restore balance relative to cold starts (see Table 3 and Figure 2). In practice the wall-clock effect depends on how targets are revised and on step size, but the dual variable provides a natural and inexpensive state to reuse.

Table 3. Adapting to revised targets: warm start versus cold start (mean $\pm$ SE over seeds).

Initialization	$\ g\ _2$	Time (s)	Iterations
Warm start	$(1.26 \pm 0.00) \times 10^{-8}$	0.401 ± 0.000	7.0 ± 0.0
Cold start	$(8.93 \pm 0.00) \times 10^{-9}$	0.397 ± 0.000	9.0 ± 0.0

6 Discussion

This paper connects entropy balancing to the multiplicative weights framework, observing that gradient descent on the well-known dual formulation induces multiplicative updates

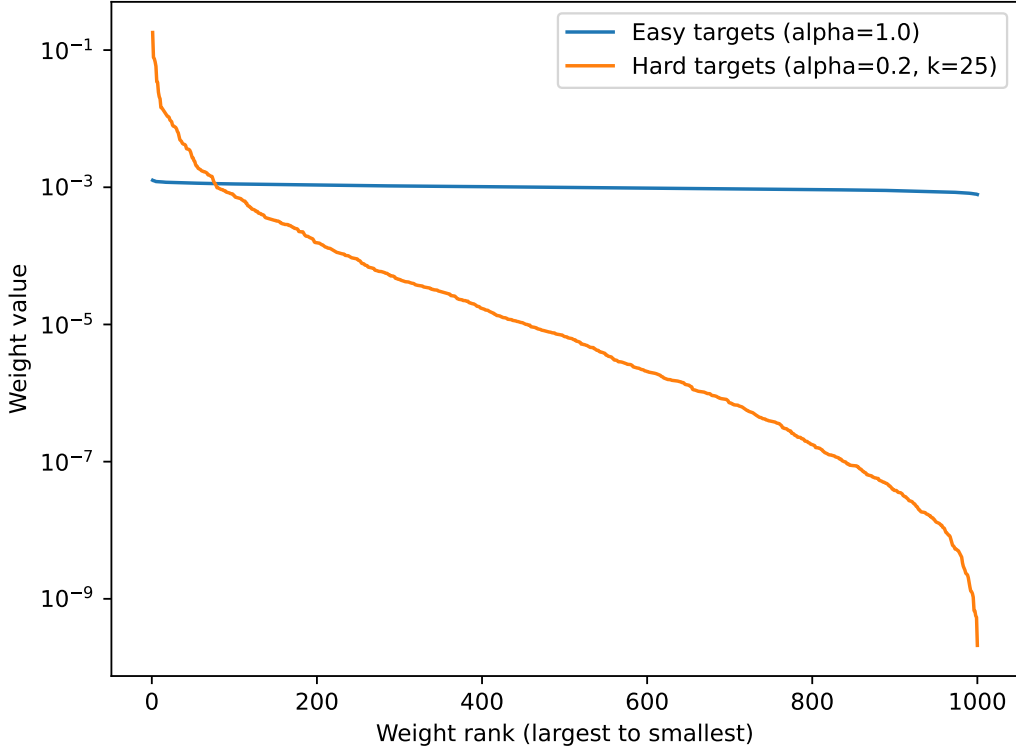


Figure 1. Weight concentration under easy versus hard targets (rank plot on a log scale).

in the primal. Closely related exponentiated and iterative-scaling perspectives appear across calibration, raking/IPF, and maximum-entropy estimation; our contribution is to make the induced multiplicative structure explicit for *dual gradient descent* on entropy balancing and to develop calibration-oriented guarantees, diagnostics, and empirical comparisons around that first-order solver.

The first-order method is most attractive when the number of constraints d is large, e.g., problems with many categories, rich interactions, or fine-grained geographic adjustments. In dense problems, each gradient step costs $O(nd)$, whereas Newton/LM-type methods typically require computations in $\mathbb{R}^{d \times d}$ (often including forming a weighted covariance at cost $O(nd^2)$ and solving a linear system at cost up to $O(d^3)$). Quasi-Newton methods reduce the need for explicit Hessians but maintain $O(d^2)$ state and typically perform multiple objective/gradient evaluations via line search. Which approach is fastest therefore depends on d , sparsity/structure of X , and the conditioning of the path induced by the targets. For small to moderate d , quasi-Newton methods remain competitive: their superlinear convergence often outweighs the per-iteration overhead, and standard implementations are well-optimized.

Warm starts are a practical advantage. Applied workflows often involve repeated

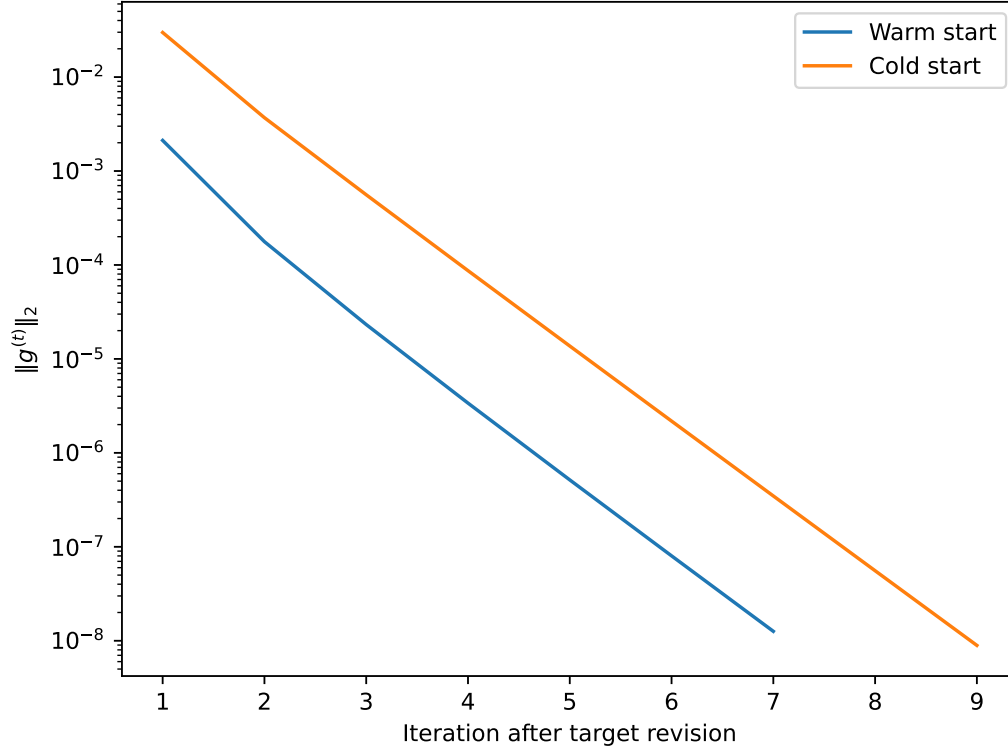


Figure 2. Warm start versus cold start after revising targets. The plot shows $\|g^{(t)}\|_2$ versus iteration after the revision.

recalibration—when benchmarks are updated, when targets are varied for sensitivity analysis, or when calibrating to sequential data releases. The dual variable λ is a compact summary of the calibration adjustment, and resuming from a previous λ typically converges in far fewer iterations than restarting. Second-order methods can also warm start, but the first-order method makes this particularly simple.

Several limitations deserve mention. First, entropy balancing requires that the moment constraints be feasible: there must exist weights in the simplex that exactly match the targets. When constraints are infeasible, for e.g., because of sparse cells, extreme targets, or collinear features, the optimization does not converge. Practitioners should check feasibility and consider relaxed or penalized formulations when exact balance is unattainable. Second, performance depends on feature scaling and conditioning. Poorly scaled features or near-collinear constraints slow convergence and may require very small step sizes. The preprocessing recommendations in [Section 4.5](#) address common cases, but some problems remain difficult. Third, the algorithm assumes access to the full sample for computing moment errors. True one-pass streaming, where observations arrive sequentially and cannot be revisited, requires approximation strategies such as reservoir sampling or cell-based summaries, which we do not develop here.

References

- Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: A meta-algorithm and applications. *Theory of Computing*, 8:121–164, 2012. doi:[10.4086/toc.2012.v008a006](https://doi.org/10.4086/toc.2012.v008a006).
- Amir Beck and Marc Teboulle. Mirror descent and nonlinear projected subgradient methods for convex optimization. *Operations Research Letters*, 31(3):167–175, 2003. doi:[10.1016/S0167-6377\(02\)00231-6](https://doi.org/10.1016/S0167-6377(02)00231-6).
- Sébastien Bubeck. Convex optimization: Algorithms and complexity. *Foundations and Trends in Machine Learning*, 8(3–4):231–357, 2015. doi:[10.1561/22000000050](https://doi.org/10.1561/22000000050).
- Imre Csiszár. I-divergence geometry of probability distributions and minimization problems. *Annals of Probability*, 3(1):146–158, 1975. doi:[10.1214/aop/1176996454](https://doi.org/10.1214/aop/1176996454).
- John N. Darroch and Douglas Ratcliff. Generalized iterative scaling for log-linear models. *The Annals of Mathematical Statistics*, 43(5):1470–1480, 1972. doi:[10.1214/aoms/1177692379](https://doi.org/10.1214/aoms/1177692379).
- W. Edwards Deming and Frederick F. Stephan. On a least squares adjustment of a sampled frequency table when the expected marginal totals are known. *The Annals of Mathematical Statistics*, 11(4):427–444, 1940. doi:[10.1214/aoms/1177731829](https://doi.org/10.1214/aoms/1177731829).
- Jean-Claude Deville and Carl-Erik Särndal. Calibration estimators in survey sampling. *Journal of the American Statistical Association*, 87(418):376–382, 1992. doi:[10.1080/01621459.1992.10475217](https://doi.org/10.1080/01621459.1992.10475217).
- Jean-Claude Deville, Carl-Erik Särndal, and Olivier Sautory. Generalized raking procedures in survey sampling. *Journal of the American Statistical Association*, 88(423):1013–1020, 1993. doi:[10.1080/01621459.1993.10476369](https://doi.org/10.1080/01621459.1993.10476369).
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(61):2121–2159, 2011. URL <https://www.jmlr.org/papers/v12/duchilla.html>.
- Jens Hainmueller. Entropy balancing for causal effects: A multivariate reweighting method to produce balanced samples in observational studies. *Political Analysis*, 20(1):25–46, 2012. doi:[10.1093/pan/mpo25](https://doi.org/10.1093/pan/mpo25).

- Jens Hainmueller and Yiqing Xu. ebalance: A stata package for entropy balancing. *Journal of Statistical Software*, 54(7):1–18, 2013. doi:[10.18637/jss.v054.i07](https://doi.org/10.18637/jss.v054.i07).
- Kosuke Imai and Marc Ratkovic. Covariate balancing propensity score. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 76(1):243–263, 2014. doi:[10.1111/rssb.12027](https://doi.org/10.1111/rssb.12027).
- C. T. Ireland and S. Kullback. Contingency tables with given marginals. *Biometrika*, 55(1):179–188, 1968. doi:[10.1093/biomet/55.1.179](https://doi.org/10.1093/biomet/55.1.179).
- Jyrki Kivinen and Manfred K. Warmuth. Exponentiated gradient versus gradient descent for linear predictors. *Information and Computation*, 132(1):1–63, 1997. doi:[10.1006/inco.1996.2612](https://doi.org/10.1006/inco.1996.2612).
- Sicheng Lin, Meng Xu, Xi Zhang, Shih-Kang Chao, Ying-Kai Huang, and Xiaolin Shi. Balancing approach for causal inference at scale. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4485–4496, 2023. doi:[10.1145/3580305.3599778](https://doi.org/10.1145/3580305.3599778).
- Yurii Nesterov. *Lectures on Convex Optimization*. Springer Optimization and Its Applications. Springer, 2 edition, 2018. doi:[10.1007/978-3-319-91578-4](https://doi.org/10.1007/978-3-319-91578-4).
- Xiaoqing Wang, Jingang Miao, and Yunting Sun. A Python library for empirical calibration. *arXiv preprint arXiv:1906.11920*, 2019. URL <https://arxiv.org/abs/1906.11920>.
- Qingyuan Zhao and Daniel Percival. Entropy balancing is doubly robust. *Journal of Causal Inference*, 5(1), 2017. doi:[10.1515/jci-2016-0010](https://doi.org/10.1515/jci-2016-0010).
- José R. Zubizarreta. Stable weights that balance covariates for estimation with incomplete outcome data. *Journal of the American Statistical Association*, 110(511):910–922, 2015. doi:[10.1080/01621459.2015.1023805](https://doi.org/10.1080/01621459.2015.1023805).

Appendix

A Proof of Proposition 1

This appendix provides a proof of Proposition 1. The dual objective is

$$D(\lambda) = \log \left(\sum_{i=1}^n u_i \exp(-\lambda^\top x_i) \right) + \lambda^\top \bar{x}_{\text{pop}},$$

and the induced weights are

$$w_i(\lambda) = \frac{u_i \exp(-\lambda^\top x_i)}{\sum_{j=1}^n u_j \exp(-\lambda^\top x_j)}.$$

Recall the moment error $g(\lambda) = X^\top w(\lambda) - \bar{x}_{\text{pop}}$.

Proof. **Step 1: Gradient and Hessian.** Differentiating $D(\lambda)$ gives

$$\nabla D(\lambda) = \bar{x}_{\text{pop}} - \sum_{i=1}^n w_i(\lambda) x_i = -g(\lambda).$$

Differentiating once more yields

$$\nabla^2 D(\lambda) = \sum_{i=1}^n w_i(\lambda) x_i x_i^\top - \left(\sum_{i=1}^n w_i(\lambda) x_i \right) \left(\sum_{i=1}^n w_i(\lambda) x_i \right)^\top = \text{Cov}_{w(\lambda)}[x].$$

In particular, $\nabla^2 D(\lambda)$ is positive semidefinite for all λ , so D is convex.

Step 2: Smoothness. Assume $\|x_i\|_2 \leq B$ for all i . For any unit vector v ,

$$v^\top \nabla^2 D(\lambda) v = \text{Var}_{w(\lambda)}(v^\top x) \leq \mathbb{E}_{w(\lambda)}[(v^\top x)^2] \leq \mathbb{E}_{w(\lambda)}[\|x\|_2^2] \leq B^2.$$

Since the Hessian is positive semidefinite, this implies $\|\nabla^2 D(\lambda)\|_{\text{op}} \leq B^2$ for all λ , and hence ∇D is L -Lipschitz with $L \leq B^2$.

Step 3: Objective-rate bound. Assume the dual optimum is attained at some λ^* .² For gradient descent with constant step size $\eta \leq 1/L$ on a convex L -smooth function, a

²If the infimum is not attained as a finite λ^* (e.g., targets on the boundary of the feasible moment set), the distance-based bound below should be interpreted conditionally on attainment; the stationarity bound in Step 4 can instead be written in terms of $D^* = \inf_{\lambda} D(\lambda)$.

standard result implies

$$D(\lambda^{(T)}) - D(\lambda^*) \leq \frac{\|\lambda^{(0)} - \lambda^*\|_2^2}{2\eta T}.$$

Setting $\eta = 1/L$ yields

$$D(\lambda^{(T)}) - D(\lambda^*) \leq \frac{L\|\lambda^{(0)} - \lambda^*\|_2^2}{2T}.$$

Step 4: Stationarity / moment-error bound. By L -smoothness, for the gradient step $\lambda^+ = \lambda - \eta \nabla D(\lambda)$ we have the descent inequality

$$D(\lambda^+) \leq D(\lambda) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla D(\lambda)\|_2^2.$$

With $\eta \leq 1/L$, the factor in parentheses is at least $1/2$, so

$$D(\lambda^{(t+1)}) \leq D(\lambda^{(t)}) - \frac{\eta}{2} \|\nabla D(\lambda^{(t)})\|_2^2.$$

Summing from $t = 0$ to $T - 1$ gives

$$\sum_{t=0}^{T-1} \|\nabla D(\lambda^{(t)})\|_2^2 \leq \frac{2}{\eta} (D(\lambda^{(0)}) - D(\lambda^{(T)})) \leq \frac{2}{\eta} (D(\lambda^{(0)}) - D(\lambda^*)).$$

Therefore,

$$\min_{0 \leq t \leq T-1} \|\nabla D(\lambda^{(t)})\|_2 \leq \sqrt{\frac{1}{T} \sum_{t=0}^{T-1} \|\nabla D(\lambda^{(t)})\|_2^2} \leq \sqrt{\frac{2(D(\lambda^{(0)}) - D(\lambda^*))}{\eta T}}.$$

Finally, since $\nabla D(\lambda) = -g(\lambda)$, the same bound holds for $\min_{t < T} \|g(\lambda^{(t)})\|_2$. □